

সিকোয়েন্স-টু-সিকোয়েন্স কাজের উদাহরণ

অনুবাদ ছাড়াও এই পদ্ধতি আরও বিভিন্ন ক্ষেত্রে ব্যবহৃত হয়, যেমন—

১. স্টাইল ট্রান্সফার (**Style Transfer**)

এক ধরনের লেখার শৈলীকে অন্য শৈলীতে রূপান্তর করা।

উদাহরণ:

- Formal English → Casual English
- Shakespearean English → Modern English

২. জেনারেটিভ প্রশ্নোত্তর (**Generative Question Answering**)

কোনো প্রেক্ষাপট (Context) দেওয়া হলে তার ভিত্তিতে প্রশ্নের উত্তর তৈরি করা।

নতুন অনুবাদ মডেল তৈরির উপায়

যদি দুই বা ততোধিক ভাষায় পর্যাপ্ত পরিমাণ ডেটাসেট থাকে, তাহলে শুরু থেকে একটি অনুবাদ মডেল প্রশিক্ষণ দেওয়া যায়। তবে এটি সময়সাপেক্ষ। তাই সাধারণত পূর্বে প্রশিক্ষিত (Pretrained) মডেলকে ফাইন-টিউন করা হয়, যা দ্রুত এবং কার্যকর।

কিছু জনপ্রিয় প্রি-ট্রেন্ড মডেল হলো—

- mT5
- mBART
- MarianMT

এগুলোকে নির্দিষ্ট ভাষা জোড়ার জন্য বা নির্দিষ্ট ডেটাসেটের উপর পুনরায় প্রশিক্ষণ দিয়ে উন্নত করা যায়।

Marian মডেল ব্যবহার

এই অংশে আমরা ইংরেজি থেকে ফরাসি অনুবাদের জন্য পূর্বপ্রশিক্ষিত **MarianMT** মডেল ব্যবহার করব। এটি Hugging Face-এর একটি জনপ্রিয় মডেল।

ব্যবহৃত মডেল:

Helsinki-NLP/opus-mt-en-fr

এই মডেলটি OPUS ডেটাসেটের বিপুল ইংরেজি ও ফরাসি টেক্সটের উপর প্রশিক্ষিত।

KDE4 ডেটাসেট

এই অধ্যায়ে আমরা **KDE4** ডেটাসেট ব্যবহার করব। এটি KDE সফটওয়্যারের লোকালাইজড ফাইলসমূহ নিয়ে তৈরি, যেখানে প্রযুক্তিগত শব্দসমূহ সঠিকভাবে অনূদিত হয়েছে।

ডেটাসেট লোড করার কোড:

```
from datasets import load_dataset
```

```
raw_datasets = load_dataset("kde4", lang1="en", lang2="fr")
```

এই ডেটাসেটে মোট ২১০,১৭৩টি বাক্যজোড়া রয়েছে।

ট্রেন ও ভ্যালিডেশন ডেটা বিভাজন

যেহেতু ডেটাসেটে একটিমাত্র অংশ রয়েছে, তাই এটিকে ট্রেন এবং ভ্যালিডেশন সেটে ভাগ করতে হয়।

```
split_datasets = raw_datasets["train"].train_test_split(  
    train_size=0.9, seed=20  
)
```

```
split_datasets["validation"] = split_datasets.pop("test")
```

বিভাজনের পর—

- Training Data: ১,৮৯,১৫৫টি বাক্যজোড়া
- Validation Data: ২১,০১৮টি বাক্যজোড়া

ডেটাসেটের একটি উদাহরণ

```
split_datasets["train"][1]["translation"]
```

আউটপুট:

```
{  
  'en': 'Default to expanded threads',  
  'fr': 'Par défaut, développer les fils de discussion'  
}
```

এখানে ইংরেজি বাক্যটি সঠিকভাবে ফরাসিতে অনূদিত হয়েছে।

প্রি-ট্রেন্ড মডেলের অনুবাদ

```
from transformers import pipeline
```

```
model_checkpoint = "Helsinki-NLP/opus-mt-en-fr"
```

```
translator = pipeline("translation", model=model_checkpoint)
```

```
translator("Default to expanded threads")
```

আউটপুট:

```
[{'translation_text': 'Par défaut pour les threads élargis'}]
```

এখানে দেখা যায়, মডেলটি “threads” শব্দটি অনুবাদ না করে ইংরেজিতেই রেখে দিয়েছে।

আরেকটি উদাহরণ: **“Plugin”**

KDE4 ডেটাসেটে “plugin” শব্দটি আনুষ্ঠানিকভাবে অনূদিত হয়েছে।

ডেটাসেট অনুবাদ:

```
{  
'en': 'Unable to import %1 using the OFX importer plugin.',  
'fr': 'Impossible d\'importer %1 en utilisant le module d\'extension d\'importation OFX.'  
}
```

কিন্তু প্রি-ট্রেন্ড মডেল অনুবাদ করে—

```
"Impossible d'importer %1 en utilisant le plugin d'importateur OFX."
```

এখানে “plugin” শব্দটি ইংরেজিতেই রাখা হয়েছে।

ফাইন-টিউন করার পরে মডেলটি ডেটাসেটের ধরণ অনুযায়ী আরও নির্ভুল অনুবাদ শিখতে পারে।

অনুশীলনী

“Email” শব্দটি ফরাসি ভাষায়ও প্রায়শই ইংরেজিতেই ব্যবহৃত হয়।

কাজ:

1. KDE4 ট্রেনিং ডেটাসেটে “email” শব্দযুক্ত প্রথম উদাহরণটি খুঁজে বের করুন।
2. দেখুন এটি কীভাবে অনূদিত হয়েছে।
3. একই বাক্য প্রি-ট্রেন্ড মডেল দিয়ে অনুবাদ করে তুলনা করুন।

ডেটা প্রসেসিং (Processing the Data)

এখন আমরা অনুবাদ মডেল প্রশিক্ষণের জন্য ডেটা প্রস্তুত করব। একটি মেশিন লার্নিং মডেল সরাসরি টেক্সট বুঝতে পারে না, তাই টেক্সটকে প্রথমে সংখ্যায় রূপান্তর করতে হয়। এই প্রক্রিয়াকে টোকেনাইজেশন (**Tokenization**) বলা হয়। এতে প্রতিটি শব্দ বা শব্দাংশকে একটি নির্দিষ্ট টোকেন আইডিতে রূপান্তর করা হয়।

এই অনুবাদ কাজের ক্ষেত্রে আমাদের ইনপুট (ইংরেজি) এবং টার্গেট (ফরাসি) — উভয় ভাষাকেই টোকেনাইজ করতে হবে।

টোকেনাইজার তৈরি করা

আমরা ইংরেজি থেকে ফরাসি অনুবাদের জন্য MarianMT প্রি-ট্রেন্ড মডেলের টোকেনাইজার ব্যবহার করব।

```
from transformers import AutoTokenizer
```

```
model_checkpoint = "Helsinki-NLP/opus-mt-en-fr"  
tokenizer = AutoTokenizer.from_pretrained(  
    model_checkpoint, return_tensors="pt"  
)
```

এখানে,

- **AutoTokenizer** স্বয়ংক্রিয়ভাবে উপযুক্ত টোকেনাইজার লোড করে।
- **return_tensors="pt"** ব্যবহারের মাধ্যমে PyTorch টেনসর তৈরি হয়।
- আপনি চাইলে Hugging Face Hub-এর অন্য কোনো মডেল বা লোকাল ফোল্ডার থেকেও টোকেনাইজার লোড করতে পারেন।
- Helsinki-NLP সংস্থা হাজারেরও বেশি ভাষার জন্য মডেল সরবরাহ করে।

বহুভাষিক টোকেনাইজারের ক্ষেত্রে নির্দেশনা

যদি mBART, mBART-50 বা M2M100-এর মতো বহুভাষিক মডেল ব্যবহার করা হয়, তাহলে উৎস এবং লক্ষ্য ভাষার কোড নির্ধারণ করতে হয়।

```
tokenizer.src_lang = "en_XX"  
tokenizer.tgt_lang = "fr_XX"
```

ইনপুট ও টার্গেট টোকেনাইজ করা

অনুবাদ মডেলের জন্য ইনপুট এবং টার্গেট উভয়কেই সঠিকভাবে টোকেনাইজ করতে হয়। এজন্য **text_target** অর্গুমেন্ট ব্যবহার করা হয়।

```
en_sentence = split_datasets["train"][1]["translation"]["en"]  
fr_sentence = split_datasets["train"][1]["translation"]["fr"]
```

```
inputs = tokenizer(en_sentence, text_target=fr_sentence)
print(inputs)
```

আউটপুট হবে এরকম:

```
{
  'input_ids': [...],
  'attention_mask': [...],
  'labels': [...]
}
```

এখানে,

- **input_ids** → ইংরেজি বাক্যের টোকেন আইডি
- **attention_mask** → গুরুত্বপূর্ণ টোকেন নির্দেশ করে
- **labels** → ফরাসি বাক্যের টোকেন আইডি

ভুলভাবে টোকেনাইজ করার উদাহরণ

যদি টার্গেট ভাষা সঠিকভাবে নির্ধারণ না করা হয়, তাহলে ভুল টোকেন তৈরি হয়।

```
wrong_targets = tokenizer(fr_sentence)
```

```
print(tokenizer.convert_ids_to_tokens(wrong_targets["input_ids"]))
print(tokenizer.convert_ids_to_tokens(inputs["labels"]))
```

এক্ষেত্রে দেখা যায়, ইংরেজি টোকেনাইজার দিয়ে ফরাসি টেক্সট প্রক্রিয়াকরণ করলে অপ্রয়োজনীয়ভাবে বেশি টোকেন তৈরি হয়। কারণ টোকেনাইজার ফরাসি শব্দগুলো সঠিকভাবে চেনে না।

প্রিপ্রসেসিং ফাংশন তৈরি

পুরো ডেটাসেটে টোকেনাইজেশন প্রয়োগ করার জন্য একটি ফাংশন তৈরি করা হয়।

```
max_length = 128
```

```
def preprocess_function(examples):
    inputs = [ex["en"] for ex in examples["translation"]]
    targets = [ex["fr"] for ex in examples["translation"]]
```

```

model_inputs = tokenizer(
    inputs,
    text_target=targets,
    max_length=max_length,
    truncation=True
)
return model_inputs

```

এখানে,

- `max_length = 128` ইনপুট ও আউটপুটের সর্বোচ্চ দৈর্ঘ্য নির্ধারণ করে।
- `truncation=True` অতিরিক্ত দীর্ঘ বাক্য কেটে দেয়।

T5 মডেলের জন্য বিশেষ নির্দেশনা

যদি T5 মডেল ব্যবহার করা হয়, তাহলে ইনপুটের শুরুতে একটি প্রিফিক্স যোগ করতে হয়।

উদাহরণ:

translate English to French:

প্যাডিং টোকেন সংক্রান্ত নির্দেশনা

মডেল প্রশিক্ষণের সময় প্যাডিং টোকেনের মান `-100` নির্ধারণ করা হয়, যাতে লস গণনার সময় সেগুলো উপেক্ষা করা হয়। এই কাজটি সাধারণত Data Collator দ্বারা স্বয়ংক্রিয়ভাবে সম্পন্ন হয়।

পুরো ডেটাসেটে প্রিপ্রসেসিং প্রয়োগ

এখন আমরা সম্পূর্ণ ডেটাসেটে প্রিপ্রসেসিং ফাংশনটি প্রয়োগ করব।

```

tokenized_datasets = split_datasets.map(
    preprocess_function,
    batched=True,
    remove_columns=split_datasets["train"].column_names,
)

```

এখানে,

- `batched=True` ব্যাচ আকারে প্রসেসিং করে গতি বাড়ায়।
- `remove_columns` অপ্রয়োজনীয় কলাম মুছে দেয়।

Trainer API ব্যবহার করে মডেল ফাইন-টিউনিং

এখন আমরা অনুবাদ মডেলকে ফাইন-টিউন করব। এই কাজের জন্য Hugging Face-এর **Trainer API** ব্যবহার করা হয়। তবে এখানে সাধারণ **Trainer**-এর পরিবর্তে **Seq2SeqTrainer** ব্যবহার করা হবে, কারণ এটি সিকোয়েন্স-টু-সিকোয়েন্স কাজের জন্য বিশেষভাবে তৈরি।

এই Trainer মূল্যায়নের সময় **generate()** মেথড ব্যবহার করে ইনপুট থেকে আউটপুট তৈরি করতে পারে, যা অনুবাদ কাজের জন্য অত্যন্ত গুরুত্বপূর্ণ।

মডেল লোড করা

প্রথমে আমাদের একটি প্রি-ট্রেন্ড মডেল লোড করতে হবে। এখানে **AutoModelForSeq2SeqLM** ব্যবহার করা হবে।

```
from transformers import AutoModelForSeq2SeqLM
```

```
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

ব্যাখ্যা:

- **AutoModelForSeq2SeqLM** সিকোয়েন্স-টু-সিকোয়েন্স ভাষা মডেলের জন্য ব্যবহৃত হয়।
- এই মডেলটি পূর্বেই অনুবাদ কাজে প্রশিক্ষিত, তাই কোনো ওজন (weights) অনুপস্থিত থাকার সতর্কবার্তা দেখা যায় না।
- এটি সরাসরি অনুবাদ করতে সক্ষম এবং ফাইন-টিউনিংয়ের মাধ্যমে আরও উন্নত করা যায়।

ডেটা কলেশন (Data Collation)

মডেল প্রশিক্ষণের সময় ডেটাকে ব্যাচ আকারে সাজানো হয়। কিন্তু প্রতিটি বাক্যের দৈর্ঘ্য সমান নয়, তাই প্যাডিং প্রয়োজন হয়। এই কাজটি Data Collator সম্পন্ন করে।

সাধারণ **DataCollatorWithPadding** এখানে ব্যবহার করা যায় না, কারণ এটি শুধু ইনপুট প্যাড করে। অনুবাদ মডেলের ক্ষেত্রে লেবেলগুলোকেও প্যাড করতে হয়।

এজন্য আমরা ব্যবহার করব **DataCollatorForSeq2Seq**।

```
from transformers import DataCollatorForSeq2Seq
```

```
data_collator = DataCollatorForSeq2Seq(  
    tokenizer=tokenizer,  
    model=model  
)
```

কেন এটি ব্যবহার করা হয়:

- ইনপুট এবং লেবেল উভয়কেই প্যাড করে।
- লেবেলের প্যাডিং মান **-100** নির্ধারণ করে, যাতে লস গণনার সময় এগুলো উপেক্ষা করা হয়।
- ডিকোডার ইনপুট আইডি স্বয়ংক্রিয়ভাবে তৈরি করে।

Data Collator পরীক্ষা করা

নিচের কোডটি কয়েকটি নমুনা নিয়ে Data Collator পরীক্ষা করে।

```
batch = data_collator(
    [tokenized_datasets["train"][i] for i in range(1, 3)]
)
```

```
batch.keys()
```

আউটপুট:

```
dict_keys(['attention_mask', 'input_ids', 'labels', 'decoder_input_ids'])
```

এখানে চারটি উপাদান রয়েছে:

- **input_ids** – ইনপুট টোকেন
- **attention_mask** – গুরুত্বপূর্ণ টোকেন নির্দেশ করে
- **labels** – টার্গেট টোকেন
- **decoder_input_ids** – ডিকোডারের ইনপুট

লেবেলের প্যাডিং যাচাই

```
batch["labels"]
```

এখানে দেখা যায়, ছোট বাক্যগুলোকে **-100** দিয়ে প্যাড করা হয়েছে। এই মানটি লস গণনার সময় উপেক্ষা করা হয়।

Decoder Input IDs যাচাই

```
batch["decoder_input_ids"]
```

এগুলো লেবেলের শিফট করা সংস্করণ। অর্থাৎ, ডিকোডার পূর্ববর্তী শব্দের ভিত্তিতে পরবর্তী শব্দ অনুমান করে।

মূল লেবেলসমূহ দেখা

```
for i in range(1, 3):
    print(tokenized_datasets["train"][i]["labels"])
```


এখানে মূল টোকেন আইডিগুলো প্রদর্শিত হয়, যা Data Collator প্যাডিং ও শিফটিংয়ের মাধ্যমে প্রশিক্ষণের উপযোগী করে।

গুরুত্বপূর্ণ ধারণা

১. Seq2SeqTrainer

- সিকোয়েন্স-টু-সিকোয়েন্স কাজের জন্য বিশেষভাবে তৈরি।
- মূল্যায়নের সময় `generate()` ব্যবহার করে অনুবাদ তৈরি করে।

২. Dynamic Padding

- প্রতিটি ব্যাচে সর্বোচ্চ দৈর্ঘ্যের ভিত্তিতে প্যাডিং করা হয়।
- এটি মেমোরি এবং গণনাশক্তি সাশ্রয় করে।

৩. Padding মান -100

- লস গণনার সময় প্যাডিং অংশ উপেক্ষা করতে ব্যবহৃত হয়।

৪. Decoder Input IDs

- লেবেলের ডানদিকে শিফট করা সংস্করণ।
- সঠিক অনুবাদ তৈরিতে সহায়তা করে।

এখন আমাদের মডেল, ডেটা এবং Data Collator প্রস্তুত। পরবর্তী ধাপে আমরা মূল্যায়নের জন্য মেট্রিক (যেমন BLEU Score) নির্ধারণ করব।

মেট্রিক্স (Metrics)

অনুবাদ মডেলের কার্যকারিতা মূল্যায়নের জন্য সঠিক মেট্রিক নির্বাচন করা অত্যন্ত গুরুত্বপূর্ণ। Seq2SeqTrainer-এর একটি বিশেষ সুবিধা হলো এটি মূল্যায়নের সময় `generate()` মেথড ব্যবহার করতে পারে, যা বাস্তব পরিস্থিতিতে মডেলের পারফরম্যান্স নির্ধারণে সহায়ক।

Seq2SeqTrainer এবং `generate()` মেথড

`Seq2SeqTrainer` সাধারণ `Trainer`-এর একটি উন্নত সংস্করণ। এটি মূল্যায়ন ও প্রেডিকশনের সময় `generate()` মেথড ব্যবহার করে।

- ট্রেনিংয়ের সময়:
মডেল `decoder_input_ids` ব্যবহার করে দ্রুত পূর্বাভাস দেয়।

- ইনফারেন্স বা মূল্যায়নের সময়:
লেবেল না থাকায় মডেল একে একে টোকেন তৈরি করে। এই প্রক্রিয়াটি `generate()` মেথডের মাধ্যমে সম্পন্ন হয়।

এই সুবিধা সক্রিয় করতে `predict_with_generate=True` সেট করতে হয়।

BLEU স্কোর

অনুবাদ মডেল মূল্যায়নের জন্য সবচেয়ে প্রচলিত মেট্রিক হলো **BLEU (Bilingual Evaluation Understudy)**। এটি ২০০২ সালে Kishore Papineni এবং তাঁর সহকর্মীরা প্রবর্তন করেন।

BLEU স্কোর যা পরিমাপ করে:

- মডেলের অনুবাদ কতটা সঠিক।
- উৎপন্ন বাক্য লক্ষ্য বাক্যের সঙ্গে কতটা মিল রয়েছে।
- শব্দের পুনরাবৃত্তি ও অপ্রয়োজনীয় সংক্ষিপ্ত বাক্যের জন্য শাস্তি প্রদান করে।

তবে BLEU সরাসরি ব্যাকরণগত শুদ্ধতা বা অর্থের গভীরতা পরিমাপ করে না; এটি পরিসংখ্যানভিত্তিক তুলনার মাধ্যমে কাজ করে।

SacreBLEU

BLEU-এর একটি সীমাবদ্ধতা হলো এটি পূর্ব-টোকেনাইজড টেক্সট প্রয়োজন করে। এই সমস্যা সমাধানের জন্য **SacreBLEU** ব্যবহার করা হয়, যা টোকেনাইজেশন প্রক্রিয়াকে মানসম্মত করে।

SacreBLEU ইনস্টলেশন

```
!pip install sacrebleu
```

SacreBLEU লোড করা

```
import evaluate
```

```
metric = evaluate.load("sacrebleu")
```

SacreBLEU ব্যবহারের নিয়ম

- **Predictions:** বাক্যের তালিকা (List of strings)
- **References:** তালিকার ভিতরে তালিকা (List of list of strings)

এটি একাধিক গ্রহণযোগ্য অনুবাদ সমর্থন করে।

উদাহরণ

```
predictions = [  
    "This plugin lets you translate web pages between several languages automatically."
```

```
]
```

```
references = [[  
    "This plugin allows you to automatically translate web pages between several languages."  
]]
```

```
metric.compute(predictions=predictions, references=references)
```

এই উদাহরণে BLEU স্কোর প্রায় 46.75, যা একটি ভালো ফলাফল।

তুলনামূলকভাবে, “Attention Is All You Need” গবেষণাপত্রে ট্রান্সফরমার মডেল ইংরেজি-ফরাসি অনুবাদে 41.8 BLEU স্কোর অর্জন করেছিল।

দুর্বল অনুবাদের উদাহরণ

অতিরিক্ত পুনরাবৃত্তি

```
predictions = ["This This This This"]
```

এক্ষেত্রে BLEU স্কোর খুব কম হয়।

অতিরিক্ত ছোট বাক্য

```
predictions = ["This plugin"]
```

এক্ষেত্রে স্কোর প্রায় শূন্য হয়।

BLEU স্কোরের পরিসীমা:

০ থেকে ১০০ — যত বেশি, তত ভালো।

Compute Metrics ফাংশন

মডেলের আউটপুটকে টেক্সটে রূপান্তর করে BLEU স্কোর গণনা করার জন্য নিচের ফাংশনটি ব্যবহার করা হয়।

```
import numpy as np
```

```
def compute_metrics(eval_preds):  
    preds, labels = eval_preds
```

```
    # যদি মডেল অতিরিক্ত তথ্য প্রদান করে
```

```
    if isinstance(preds, tuple):
```

```
        preds = preds[0]
```

```
    # প্রেডিকশন ডিকোড করা
```

```

decoded_preds = tokenizer.batch_decode(
    preds, skip_special_tokens=True
)

# -100 মানগুলোকে pad_token দিয়ে প্রতিস্থাপন
labels = np.where(
    labels != -100,
    labels,
    tokenizer.pad_token_id
)

# লেবেল ডিকোড করা
decoded_labels = tokenizer.batch_decode(
    labels, skip_special_tokens=True
)

# অপ্রয়োজনীয় স্পেস অপসারণ
decoded_preds = [pred.strip() for pred in decoded_preds]
decoded_labels = [[label.strip()] for label in decoded_labels]

# SacreBLEU স্কোর গণনা
result = metric.compute(
    predictions=decoded_preds,
    references=decoded_labels
)

return {"bleu": result["score"]}

```

ফাংশনটির কার্যপ্রণালি

ধাপ	ব্যাখ্যা
Predictions Decode	টোকেন আইডিকে টেক্সটে রূপান্তর করে
Labels Decode	সঠিক অনুবাদ টেক্সটে রূপান্তর করে
-100 প্রতিস্থাপন	প্যাডিং টোকেনকে উপেক্ষা করতে ব্যবহৃত হয়
Post-processing	অতিরিক্ত স্পেস অপসারণ করে
SacreBLEU গণনা	অনুবাদের মান নির্ধারণ করে

মডেল ফাইন-টিউনিং (Fine-Tuning the Model)

এখন আমরা অনুবাদ মডেলটি সম্পূর্ণভাবে ফাইন-টিউন করব। এই ধাপে Hugging Face Hub-এ লগইন, ট্রেনিং আর্গুমেন্ট নির্ধারণ, মডেল প্রশিক্ষণ, মূল্যায়ন এবং মডেল আপলোড করা হবে।

Hugging Face-এ লগইন

মডেলকে Model Hub-এ আপলোড করার জন্য প্রথমে Hugging Face-এ লগইন করতে হবে।

Notebook ব্যবহার করলে

```
from huggingface_hub import notebook_login
```

```
notebook_login()
```

এটি একটি উইজেট প্রদর্শন করবে, যেখানে আপনার Hugging Face অ্যাকাউন্টের তথ্য দিতে হবে।

Terminal ব্যবহার করলে

```
huggingface-cli login
```

Seq2SeqTrainingArguments নির্ধারণ

এরপর ট্রেনিংয়ের জন্য প্রয়োজনীয় হাইপারপ্যারামিটার নির্ধারণ করা হবে। Seq2SeqTrainer ব্যবহারের জন্য **Seq2SeqTrainingArguments** ব্যবহার করা হয়।

```
from transformers import Seq2SeqTrainingArguments
```

```
args = Seq2SeqTrainingArguments(  
    "marian-finetuned-kde4-en-to-fr",  
    evaluation_strategy="no",  
    save_strategy="epoch",  
    learning_rate=2e-5,  
    per_device_train_batch_size=32,  
    per_device_eval_batch_size=64,  
    weight_decay=0.01,  
    save_total_limit=3,  
    num_train_epochs=3,  
    predict_with_generate=True,  
    fp16=True,  
    push_to_hub=True,  
)
```

গুরুত্বপূর্ণ প্যারামিটারসমূহ

প্যারামিটার	ব্যাখ্যা
learning_rate	মডেলের শেখার গতি নিয়ন্ত্রণ করে
num_train_epochs	মোট প্রশিক্ষণ চক্রের সংখ্যা
per_device_train_batch_size	প্রতিটি ডিভাইসে ট্রেনিং ব্যাচের আকার
per_device_eval_batch_size	মূল্যায়নের ব্যাচ সাইজ
weight_decay	ওভারফিটিং কমাতে সাহায্য করে
save_strategy="epoch"	প্রতিটি epoch শেষে মডেল সংরক্ষণ করে
predict_with_generate=True	মূল্যায়নের সময় <code>generate()</code> ব্যবহার করে
fp16=True	আধুনিক GPU-তে দ্রুত প্রশিক্ষণ নিশ্চিত করে
push_to_hub=True	মডেল Hugging Face Hub-এ আপলোড করে

ডিফল্টভাবে রিপোজিটরিটি আপনার ইউজারনেমের অধীনে তৈরি হবে।

Seq2SeqTrainer তৈরি

এখন মডেল, ডেটাসেট এবং অন্যান্য উপাদান Trainer-এ প্রদান করা হবে।

```
from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    model=model,
    args=args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
)
```

প্রশিক্ষণের আগে মূল্যায়ন

ফাইন-টিউনিং শুরু করার আগে মডেলের প্রাথমিক পারফরম্যান্স যাচাই করা হয়।

```
trainer.evaluate(max_length=max_length)
```

উদাহরণস্বরূপ আউটপুট:

```
{  
'eval_loss': 1.6964,  
'eval_bleu': 39.27,  
'eval_runtime': 965.88,  
}
```

ব্যাখ্যা:

BLEU স্কোর 39 হওয়া মানে মডেলটি ইতোমধ্যে ভালো অনুবাদ করতে সক্ষম।

মডেল প্রশিক্ষণ

```
trainer.train()
```

- প্রশিক্ষণের সময় প্রতিটি epoch শেষে মডেল সংরক্ষণ করা হয়।
- `push_to_hub=True` থাকলে মডেল স্বয়ংক্রিয়ভাবে Hugging Face Hub-এ আপলোড হয়।
- প্রয়োজনে অন্য কম্পিউটার থেকে পুনরায় প্রশিক্ষণ চালানো যায়।

প্রশিক্ষণের পর মূল্যায়ন

```
trainer.evaluate(max_length=max_length)
```

উদাহরণস্বরূপ আউটপুট:

```
{  
'eval_loss': 0.8558,  
'eval_bleu': 52.94,  
'epoch': 3.0  
}
```

এখানে BLEU স্কোর প্রায় ১৪ পয়েন্ট বৃদ্ধি পেয়েছে, যা উল্লেখযোগ্য উন্নতি নির্দেশ করে।

মডেল **Hub**-এ আপলোড করা

```
trainer.push_to_hub(  
    tags="translation",  
    commit_message="Training complete"  
)
```

এই কমান্ডটি:

- মডেলের সর্বশেষ সংস্করণ আপলোড করে।
- স্বয়ংক্রিয়ভাবে একটি Model Card তৈরি করে।
- ইনফারেন্স ডেমোর জন্য প্রয়োজনীয় মেটাডেটা যুক্ত করে।

উদাহরণস্বরূপ একটি কমিট লিংক:

<https://huggingface.co/username/marian-finetuned-kde4-en-to-fr>

কাস্টম ট্রেনিং লুপ (A Custom Training Loop)

এখন আমরা সম্পূর্ণ ট্রেনিং লুপটি দেখব, যাতে আপনি আপনার প্রয়োজন অনুযায়ী কোড সহজেই পরিবর্তন করতে পারেন। এটি পূর্ববর্তী অধ্যায়গুলোর মতো হলেও এখানে Accelerate ব্যবহার করা হয়েছে, যা দ্রুত ও দক্ষভাবে ট্রেনিং সম্পন্ন করতে সাহায্য করে।

ট্রেনিংয়ের জন্য প্রস্তুতি

প্রথমে ডেটাসেটকে PyTorch টেনসর ফরম্যাটে রূপান্তর করে DataLoader তৈরি করা হবে।

```
from torch.utils.data import DataLoader
```

```
tokenized_datasets.set_format("torch")
```

```
train_dataloader = DataLoader(  
    tokenized_datasets["train"],  
    shuffle=True,  
    collate_fn=data_collator,  
    batch_size=8,  
)
```

```
eval_dataloader = DataLoader(  
    tokenized_datasets["validation"],  
    collate_fn=data_collator,  
    batch_size=8,  
)
```

ব্যাখ্যা:

- `set_format("torch")` ডেটাকে PyTorch টেনসরে রূপান্তর করে।
- `train_dataloader` প্রশিক্ষণের জন্য ব্যবহৃত হয়।
- `eval_dataloader` মূল্যায়নের জন্য ব্যবহৃত হয়।
- `data_collator` ডায়নামিক প্যাডিং নিশ্চিত করে।

মডেল পুনরায় লোড করা

ফাইন-টিউনিং নতুন করে শুরু করার জন্য প্রি-ট্রেন্ড মডেল পুনরায় লোড করা হয়।

```
from transformers import AutoModelForSeq2SeqLM
```

```
model = AutoModelForSeq2SeqLM.from_pretrained(model_checkpoint)
```

অপ্টিমাইজার নির্ধারণ

```
from torch.optim import AdamW
```

```
optimizer = AdamW(model.parameters(), lr=2e-5)
```

এটি মডেলের ওজন আপডেট করার জন্য ব্যবহৃত হয়।

Accelerate ব্যবহার

```
from accelerate import Accelerator
```

```
accelerator = Accelerator()
```

```
model, optimizer, train_dataloader, eval_dataloader = accelerator.prepare(
    model, optimizer, train_dataloader, eval_dataloader
)
```

উপকারিতা:

- GPU, TPU এবং মাল্টি-GPU সাপোর্ট করে।
- দ্রুত ও দক্ষ ট্রেনিং নিশ্চিত করে।

লার্নিং রেট স্কেডিউলার

```
from transformers import get_scheduler

num_train_epochs = 3
num_update_steps_per_epoch = len(train_dataloader)
num_training_steps = num_train_epochs * num_update_steps_per_epoch

lr_scheduler = get_scheduler(
    "linear",
    optimizer=optimizer,
    num_warmup_steps=0,
    num_training_steps=num_training_steps,
)
```

এটি ধীরে ধীরে লার্নিং রেট কমায়।

Hugging Face Hub-এ রিপোজিটরি তৈরি

```
from huggingface_hub import Repository, get_full_repo_name

model_name = "marian-finetuned-kde4-en-to-fr-accelerate"
repo_name = get_full_repo_name(model_name)

output_dir = "marian-finetuned-kde4-en-to-fr-accelerate"
repo = Repository(output_dir, clone_from=repo_name)
```

এটি মডেল সংরক্ষণ ও আপলোড করতে ব্যবহৃত হয়।

পোস্ট-প্রসেসিং ফাংশন

```
import numpy as np

def postprocess(predictions, labels):
    predictions = predictions.cpu().numpy()
    labels = labels.cpu().numpy()
```

```

decoded_preds = tokenizer.batch_decode(
    predictions, skip_special_tokens=True
)

labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
decoded_labels = tokenizer.batch_decode(
    labels, skip_special_tokens=True
)

decoded_preds = [pred.strip() for pred in decoded_preds]
decoded_labels = [[label.strip()] for label in decoded_labels]

return decoded_preds, decoded_labels

```

এটি BLEU স্কোর গণনার জন্য টেক্সট প্রস্তুত করে।

সম্পূর্ণ ট্রেনিং লুপ

```

from tqdm.auto import tqdm
import torch

progress_bar = tqdm(range(num_training_steps))

for epoch in range(num_train_epochs):
    # Training
    model.train()
    for batch in train_dataloader:
        outputs = model(**batch)
        loss = outputs.loss
        accelerator.backward(loss)

        optimizer.step()
        lr_scheduler.step()
        optimizer.zero_grad()
        progress_bar.update(1)

    # Evaluation
    model.eval()
    for batch in tqdm(eval_dataloader):
        with torch.no_grad():
            generated_tokens = accelerator.unwrap_model(model).generate(

```

```

        batch["input_ids"],
        attention_mask=batch["attention_mask"],
        max_length=128,
    )

    labels = batch["labels"]

    generated_tokens = accelerator.pad_across_processes(
        generated_tokens,
        dim=1,
        pad_index=tokenizer.pad_token_id,
    )

    labels = accelerator.pad_across_processes(
        labels,
        dim=1,
        pad_index=-100,
    )

    predictions_gathered = accelerator.gather(generated_tokens)
    labels_gathered = accelerator.gather(labels)

    decoded_preds, decoded_labels = postprocess(
        predictions_gathered, labels_gathered
    )

    metric.add_batch(
        predictions=decoded_preds,
        references=decoded_labels,
    )

    results = metric.compute()
    print(f"epoch {epoch}, BLEU score: {results['score']:.2f}")

# Save and Upload
accelerator.wait_for_everyone()
unwrapped_model = accelerator.unwrap_model(model)
unwrapped_model.save_pretrained(
    output_dir,
    save_function=accelerator.save,
)

if accelerator.is_main_process:
    tokenizer.save_pretrained(output_dir)

```

```
repo.push_to_hub(  
    commit_message=f"Training in progress epoch {epoch}",  
    blocking=False,  
)
```

উদাহরণ আউটপুট

epoch 0, BLEU score: 53.47

epoch 1, BLEU score: 54.24

epoch 2, BLEU score: 54.44

এটি দেখায় যে প্রতিটি epoch-এর সাথে মডেলের কর্মক্ষমতা উন্নত হচ্ছে।

ফাইন-টিউনড মডেল ব্যবহার

লোকালভাবে অনুবাদ করতে নিচের কোড ব্যবহার করা যায়।

```
from transformers import pipeline
```

```
model_checkpoint = "huggingface-course/marian-finetuned-kde4-en-to-fr"
```

```
translator = pipeline("translation", model=model_checkpoint)
```

```
translator("Default to expanded threads")
```

আউটপুট:

Par défaut, développer les fils de discussion

আরেকটি উদাহরণ:

```
translator(  
    "Unable to import %1 using the OFX importer plugin. This file is not the correct format."  
)
```

আউটপুট:

Impossible d'importer %1 en utilisant le module externe d'importation OFX. Ce fichier n'est pas le bon format.

এটি প্রমাণ করে যে মডেলটি নির্দিষ্ট ডোমেইনের সাথে মানিয়ে নিতে সক্ষম হয়েছে।

সংক্ষিপ্তসার

ধাপ	বর্ণনা
DataLoader	ট্রেনিং ও মূল্যায়নের জন্য ডেটা প্রস্তুত করে
AutoModelForSeq2SeqLM	প্রি-ট্রেন্ডেড অনুবাদ মডেল লোড করে
AdamW	অপটিমাইজার হিসেবে ব্যবহৃত হয়
Accelerator	দ্রুত ও দক্ষ ট্রেনিং নিশ্চিত করে
Scheduler	লার্নিং রেট নিয়ন্ত্রণ করে
Custom Training Loop	সম্পূর্ণ নিয়ন্ত্রণ প্রদান করে
BLEU Score	অনুবাদের মান মূল্যায়ন করে
Hugging Face Hub	মডেল সংরক্ষণ ও শেয়ার করে